Звіт по проєкту «AutoMarket — дошка оголошень б/у техніки та авто»

Тема проєкту: вебсервіс для публікації оголошень про продаж вживаної техніки та автомобілів із модерацією оголошень адміністратором.

Стек технологій: ASP.NET Core 8 Web API (REST), Entity Framework Core 8 (code-first, міграції), SQLite, JWT-автентифікація з ролями User/Admin, Angular 22 + TypeScript, xUnit.

1. Загальний опис системи

Призначення: система дозволяє швидко та безпечно продати вживану річ. Кожне оголошення перед публікацією перевіряє адміністратор, тому в каталозі немає спаму та фейкових оголошень.

Ролі користувачів:

- **Гість** – переглядає каталог активних оголошень, користується пошуком і фільтрами, відкриває сторінку оголошення, може зареєструватися.
- **Користувач** – створює та редагує власні оголошення, бачить їхні статуси та причину відхилення, додає оголошення в обране, змінює профіль і пароль.
- **Адміністратор** – схвалює або відхиляє оголошення, блокує та видаляє користувачів, змінює ролі, керує категоріями, переглядає статистику.

Структура рішення (AutoMarket.sln):

- `AutoMarket.Api` – серверна частина, REST API на ASP.NET Core;
- `AutoMarket.Web` – клієнтська частина, односторінковий застосунок на Angular;
- `AutoMarket.Tests` – модульні та інтеграційні тести (xUnit);
- `docs` – діаграми, презентація та цей звіт.

2. Архітектура системи

Тип архітектури: клієнт-серверна. Клієнт (Angular) спілкується з сервером лише через REST API у форматі JSON; сервер розділено на шари.

Шари серверної частини:

- **Middleware** – конвеєр обробки запитів: логування, обробка помилок, CORS, обмеження частоти запитів, JWT-автентифікація, перевірка заблокованих користувачів, авторизація;

- **Controllers** – 8 «тонких» контролерів, які лише приймають запит і викликають сервіс;
- **Services** – уся бізнес-логіка (8 сервісів, кожен з інтерфейсом для тестування та DI);
- **Data** – `AppDbContext` (EF Core), міграції та `DbSeeder` із початковими даними.

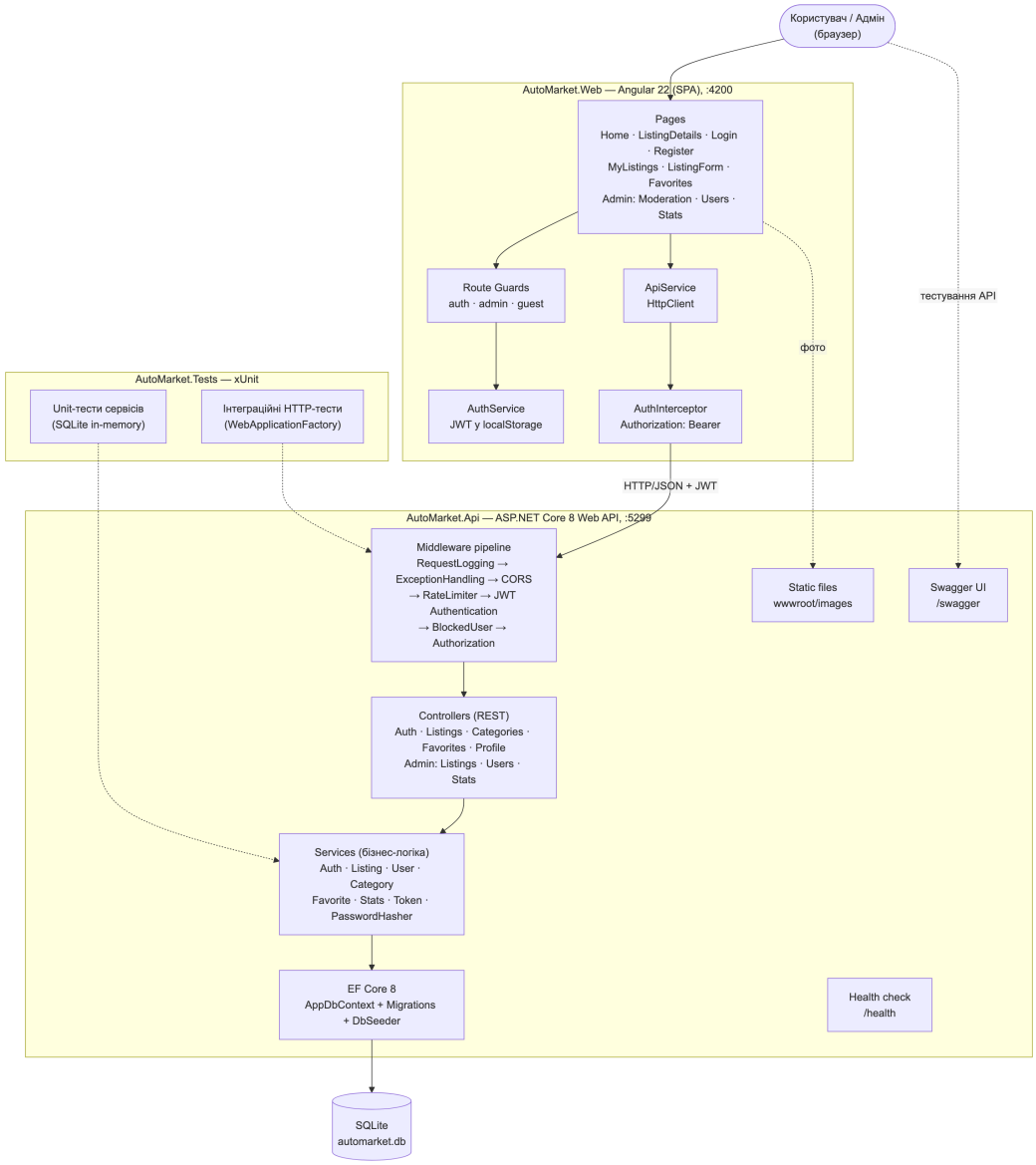


Рисунок 1 – Схема архітектури AutoMarket

Порядок middleware в Program.cs:

```
app.UseMiddleware<RequestLoggingMiddleware>();
app.UseMiddleware<ExceptionHandlingMiddleware>();
app.UseStaticFiles();
app.UseSwagger();
app.UseCors(ServiceCollectionExtensions.CorsPolicy);
app.UseRateLimiter();
app.UseAuthentication();
app.UseMiddleware<BlockedUserMiddleware>();
app.UseAuthorization();
```

3. Модель даних

Підхід: code-first – спочатку описуються C#-класи моделей, а таблиці створюються міграціями EF Core (`dotnet ef migrations add`). Міграції застосовуються автоматично під час запуску.

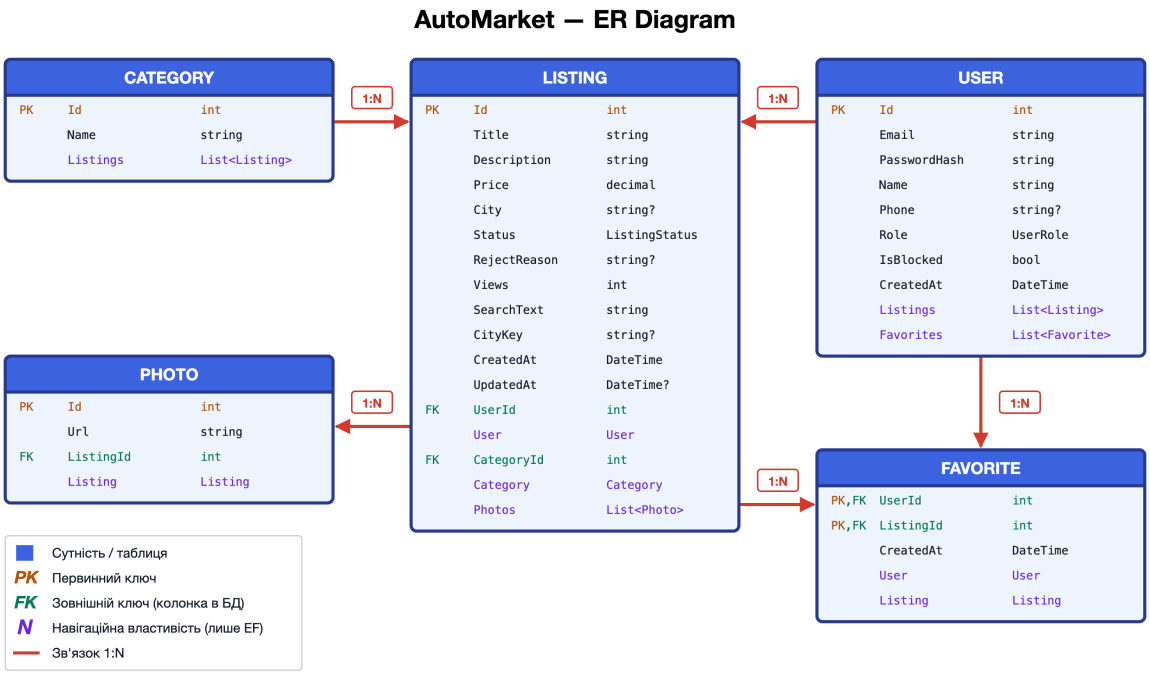


Рисунок 2 – ER-діаграма бази даних

Сутності:

- **User** – користувач: email (унікальний), хеш пароля BCrypt, ім'я, телефон, роль, ознака блокування;
- **Category** – категорія оголошень (6 категорій у початкових даних);
- **Listing** – оголошення: назва, опис, ціна, місто, статус, причина відхилення, кількість переглядів, дати створення та зміни;
- **Photo** – фото оголошення (зберігається посилання);
- **Favorite** – обране, зв'язок «багато-до-багатьох» між User і Listing зі складеним ключем (UserId, ListingId).

Статуси оголошення (enum ListingStatus):

```
public enum ListingStatus
{
    Pending,    // очікує модерації (після створення або редагування)
    Active,     // схвалене, видно всім
    Rejected,   // відхилене адміністратором (з причиною)
    Deleted     // м'яке видалення, запис лишається в БД
}
```

Особливості налаштування контексту:

- статус і роль зберігаються в БД рядками (`HasConversion<string>()`), а не числами – так таблиці легше читати;
- для регістронезалежного пошуку кирилицею поля `SearchText` і `CityKey` заповнюються автоматично перед кожним збереженням (SQLite-функція lower() працює лише з латиницею);
- видалення категорії, що містить оголошення, заборонене (`DeleteBehavior.Restrict`).

4. Серверна частина (AutoMarket.Api)

4.1. Автентифікація (AuthService, TokenService)

Призначення: реєстрація, вхід і видача JWT-токена з роллю користувача.

Логіка роботи:

- email перед збереженням приводиться до нижнього регістру, повторна реєстрація повертає 409 Conflict;
- пароль зберігається лише у вигляді хешу BCrypt із сіллю;
- для невідомого email і неправильного пароля повертається однакова помилка 401, щоб не підказувати, які email існують;
- заблокований користувач отримує 403 Forbidden.

Вміст токена:

```
var claims = new[]
{
    new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
    new Claim(ClaimTypes.Email, user.Email),
    new Claim(ClaimTypes.Name, user.Name),
    new Claim(ClaimTypes.Role, user.Role.ToString())
};
```

4.2. Оголошення та модерація (ListingService)

Призначення: каталог для гостей, кабінет користувача та модерація адміністратором.

Каталог:

- показуються лише оголошення зі статусом Active;
- пошук за текстом, фільтри за категорією, ціною (від/до) і містом;
- сортування: нові, старі, дешеві, дорогі, популярні;
- пагінація (розмір сторінки 1–100), відповідь містить TotalCount, TotalPages, HasNext;
- перегляд оголошення збільшує лічильник переглядів.

Правила зміни статусу:

- нове оголошення завжди отримує статус Pending;
- будь-яке редагування повертає оголошення на модерацію:

```
Apply(listing, dto);
db.Photos.RemoveRange(listing.Photos);
listing.Photos = ToPhotos(dto.PhotoUrls);
// будь-яка зміна повертає оголошення на модерацію
listing.Status = ListingStatus.Pending;
listing.RejectReason = null;
listing.UpdatedAt = DateTime.UtcNow;
```

- відхилення без причини заборонене, видалене оголошення не можна схвалити;
- редагувати й видаляти можна лише власні оголошення (інакше 403).

Перевірки фото: до 10 посилань, лише http/https або локальні файли демоданих, дублікати відкидаються.

4.3. Користувачі, категорії, обране, статистика

- **UserService** – профіль, зміна пароля, список користувачів з пошуком, блокування, зміна ролі, видалення. Адміністратор не може заблокувати, видалити або понизити себе, а останнього адміністратора видалити неможливо;
- **CategoryService** – CRUD категорій із перевіркою унікальності назви;
- **FavoriteService** – обране; повторне додавання не є помилкою, власне оголошення додати не можна;
- **StatsService** – кількість користувачів, оголошень за статусами та категоріями, середня ціна.

4.4. Middleware та обробка помилок

ExceptionHandlingMiddleware: перетворює бізнес-виключення (NotFoundException, ForbiddenException тощо) на HTTP-відповідь у

стандартному форматі ProblemDetails (RFC 7807). Невідомі помилки повертаються як 500 без деталей стеку.

BlockedUserMiddleware: JWT діє 7 днів, тому після блокування старий токен ще валідний. Middleware перевіряє користувача в БД на кожному авторизованому запиті:

```
if (context.User.Identity?.IsAuthenticated == true)
{
    var userId = context.User.GetUserId();
    var state = await db.Users.AsNoTracking().Where(u => u.Id == userId)
        .Select(u => new { u.IsBlocked }).FirstOrDefaultAsync();

    if (state == null)
        throw new UnauthorizedException("Користувача більше не існує");
    if (state.IsBlocked)
        throw new ForbiddenException("Користувача заблоковано
адміністратором");
}
```

RequestLoggingMiddleware: логує метод, шлях, статус і час виконання кожного запиту та додає заголовок X-Request-Id.

5. REST API

API містить 32 ендпоінти у 8 контролерах. Усі методи задокументовані XML-коментарями й доступні для тестування у Swagger UI за адресою `/swagger`. Помилки повертаються у форматі ProblemDetails, вхідні дані перевіряються атрибутами DataAnnotations.

Таблиця 1 – Основні ендпоінти API

Метод	Шлях	Доступ	Опис
POST	`/api/auth/register`	Гість	Реєстрація
POST	`/api/auth/login`	Гість	Вхід, отримання JWT
GET	`/api/listings`	Гість	Каталог: пошук, фільтри, пагінація
GET	`/api/listings/{id}`	Гість	Активне оголошення
GET	`/api/listings/my`	User	Мої оголошення
GET	`/api/listings/my/{id}`	User	Моє оголошення (будь-який статус)
POST	`/api/listings`	User	Створити оголошення
PUT	`/api/listings/{id}`	User	Редагувати своє оголошення
DELETE	`/api/listings/{id}`	User	Видалити своє оголошення
GET	`/api/categories`	Гість	Список категорій

Метод	Шлях	Доступ	Опис
GET	`/api/categories/{id}`	Гість	Категорія
POST / PUT / DELETE	`/api/categories/{id}`	Admin	Керування категоріями
GET / POST / DELETE	`/api/favorites/{listingId}`	User	Обране
GET / PUT	`/api/profile`	User	Профіль
POST	`/api/profile/change-password`	User	Зміна пароля
GET	`/api/admin/listings`	Admin	Черга модерації
POST	`/api/admin/listings/{id}/approve`	Admin	Схвалити
POST	`/api/admin/listings/{id}/reject`	Admin	Відхилити з причиною
DELETE	`/api/admin/listings/{id}`	Admin	Видалити будь-яке оголошення
GET	`/api/admin/users`	Admin	Користувачі з пошуком
GET	`/api/admin/users/{id}`	Admin	Користувач і його оголошення
POST	`/api/admin/users/{id}/block`	Admin	Заблокувати / розблокувати
PUT	`/api/admin/users/{id}/role`	Admin	Змінити роль
DELETE	`/api/admin/users/{id}`	Admin	Видалити користувача
GET	`/api/admin/stats`	Admin	Статистика

Приклад запиту до каталогу:

```
GET /api/listings?q=toyota&categoryId=1&sort=PriceAsc

200 OK
{
  "items": [
    { "title": "Toyota Camry Hybrid 2012", "price": 485000, ... },
    { "title": "Toyota RAV4 XLE AWD", "price": 640000, ... }
  ],
  "page": 1,
  "totalCount": 2,
  "hasNext": false
}
```

6. Клієнтська частина (AutoMarket.Web)

Технології: Angular 22, standalone-компоненти, signals, ліниве завантаження сторінок (lazy loading), HttpClient.

Маршрути (app.routes.ts):

- **Публічні:**
  - `/` – головна сторінка з каталогом;
  - `/listing/:id` – перегляд оголошення;
- **Лише для гостей** (guestGuard): `/login`, `/register`;
- **Для авторизованих** (authGuard):
  - `/my` – мої оголошення з вкладками Активні / На модерації / Відхилені;
  - `/create` і `/edit/:id` – створення та редагування оголошення;
  - `/favorites` – обране;
- **Адміністративні** (adminGuard): `/admin/moderation`, `/admin/users`, `/admin/stats`;
- `**` – сторінка 404 для неіснуючих маршрутів.

Захист маршрутів (guards.ts):

```
/** Лише для авторизованих, інакше — на сторінку входу з поверненням
назад. */
export const authGuard: CanActivateFn = (_route, state) => {
  const auth = inject(AuthService);
  return auth.isLoggedIn() || inject(Router).createUrlTree(['/login'],
    { queryParams: { returnUrl: state.url } });
};
```

Робота з токеном:

- AuthService зберігає відповідь сервера в localStorage, протермінований токен відкидається при завантаженні;
- authInterceptor додає заголовок `Authorization: Bearer <token>` до кожного запиту, а при відповіді 401 автоматично виконує вихід;
- ApiService містить окремий метод для кожного ендпоінта та перетворює помилки сервера на зрозумілий текст.

Інтерфейс:

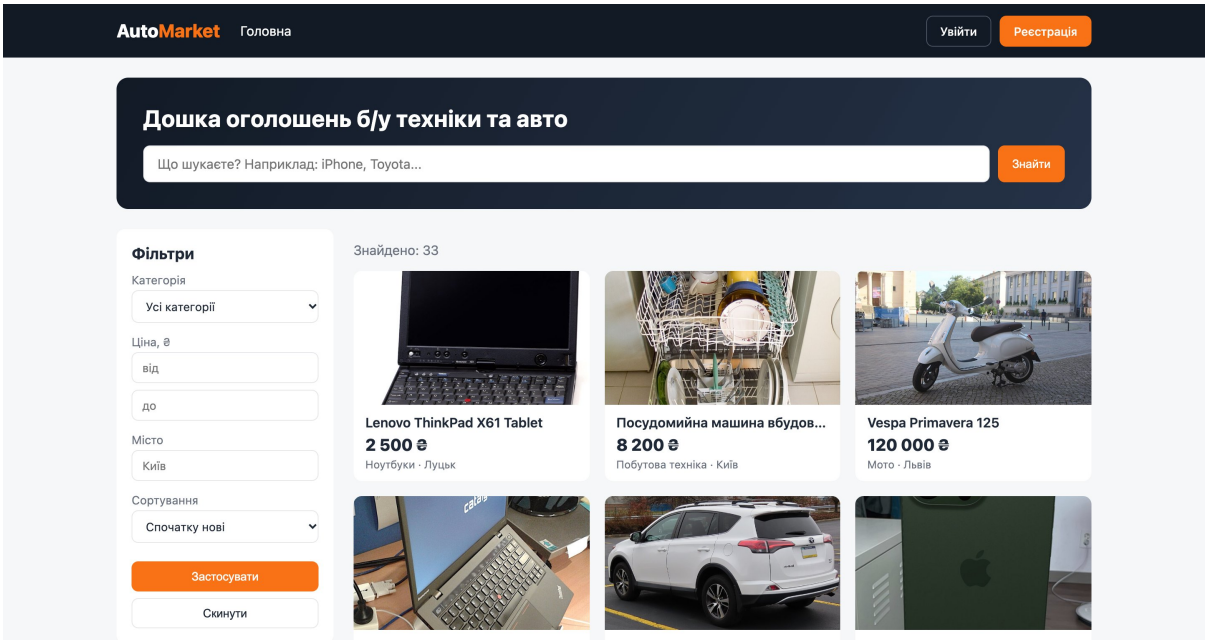


Рисунок 3 – Головна сторінка: пошук, фільтри та каталог

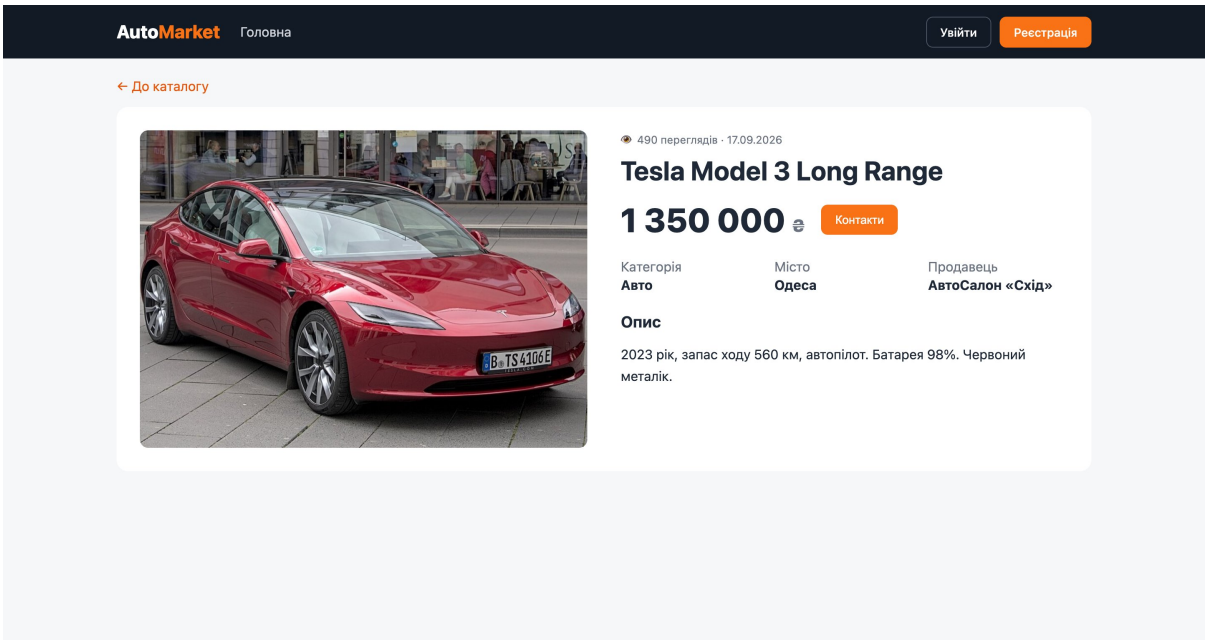


Рисунок 4 – Сторінка оголошення

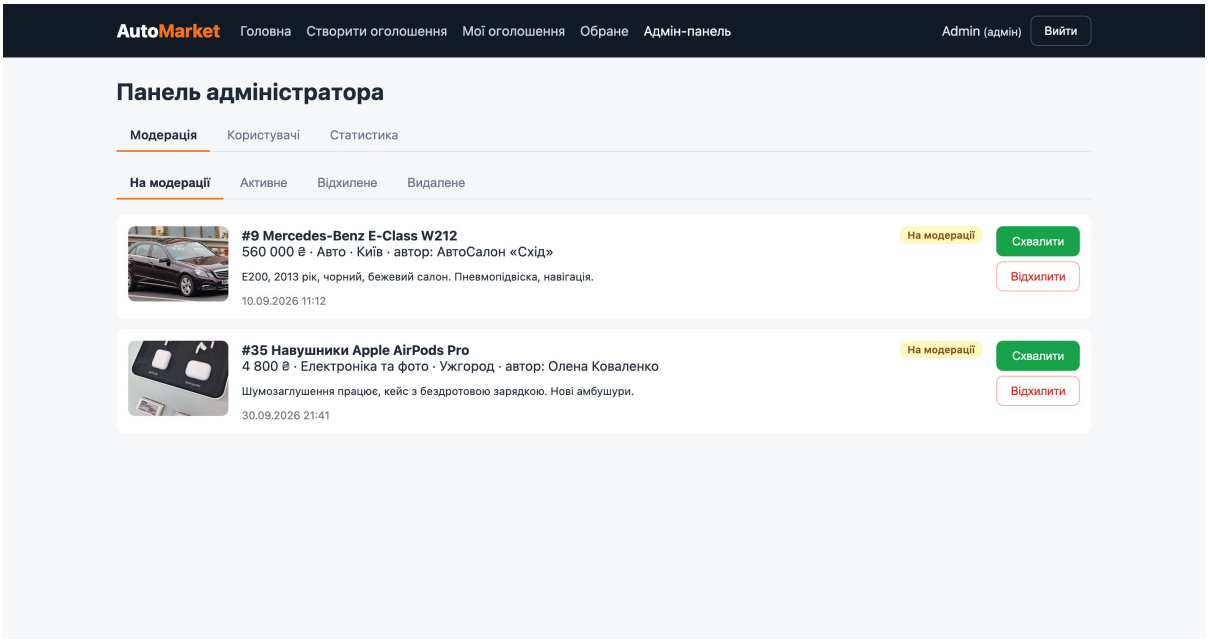


Рисунок 5 – Панель адміністратора: модерація оголошень

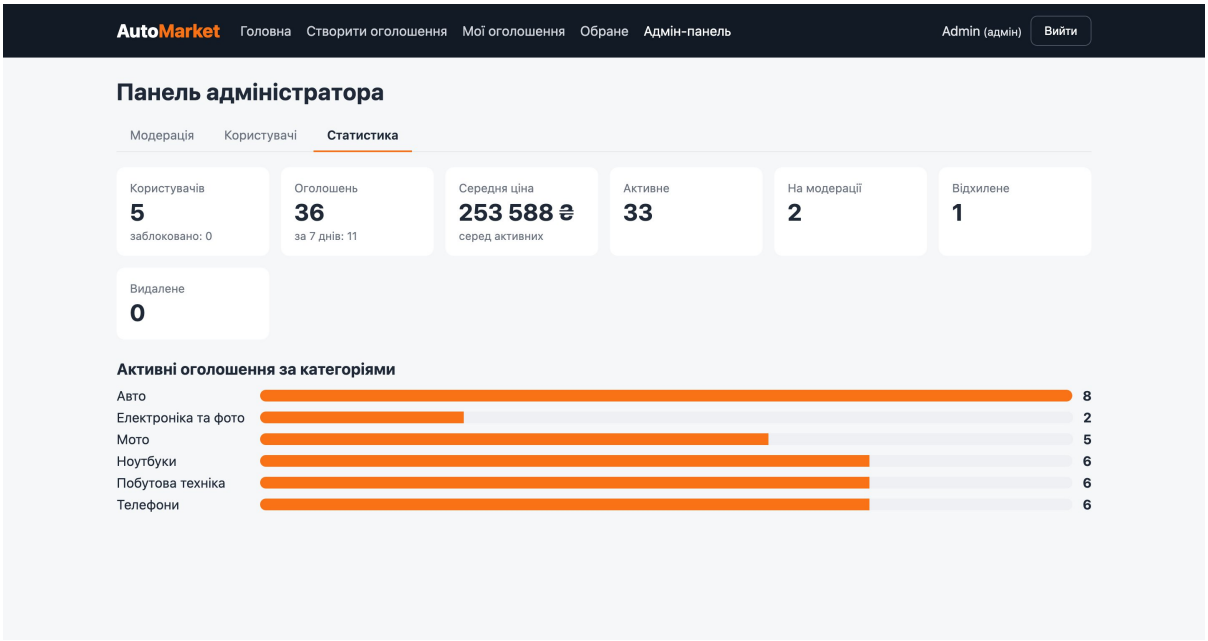


Рисунок 6 – Панель адміністратора: статистика

7. Безпека

- JWT-токени з перевіркою підпису, видавця, отримувача та терміну дії;
- ролі User та Admin, адмінські ендпоінти закриті атрибутом `[Authorize(Roles = "Admin")]`;
- паролі зберігаються лише як хеш BCrypt;
- блокування користувача діє миттєво навіть для вже виданих токенів;
- обмеження частоти запитів до `/api/auth` (захист від перебору паролів);

- валідація всіх вхідних даних і захист від підстановки шляхів у посиланнях на фото;
- користувач не може змінювати чужі оголошення, адміністратор – видалити або заблокувати себе.

8. Тестування

Для перевірки працездатності створено окремий проєкт `AutoMarket.Tests` (xUnit + FluentAssertions). Усього **109 тестів**, усі проходять.

Таблиця 2 – Тести проєкту

Вид тестів	Кількість	Що перевіряють
Модульні (Unit)	83	Сервіси на справжній SQLite у пам'яті: реєстрація, вхід, пошук, фільтри, сортування, пагінація, статуси, права доступу, обране, статистика
Інтеграційні	26	Увесь API через HTTP (WebApplicationFactory): коди 401/403, валідація, повний сценарій «створення → модерація → пошук → обране», блокування

Помилки, знайдені тестами та виправлені:

- пошук не знаходив кириличні слова без урахування регістру (обмеження SQLite) – додано нормалізовані поля для пошуку;
- помилка побудови запиту у списку обраного (Include після Select);
- заблокований користувач міг працювати зі старим токеном – додано BlockedUserMiddleware;
- дати поверталися без часового поясу й відображалися на 3 години раніше – дати позначаються як UTC.

9. Запуск проєкту

API: відкрити `AutoMarket.sln` у JetBrains Rider, вибрати конфігурацію `AutoMarket.Api: http` і натиснути Run. База даних створюється автоматично, Swagger відкривається за адресою http://localhost:5299/swagger.

Вебінтерфейс: у терміналі виконати `cd AutoMarket.Web`, потім `npm install` та `npm start`. Сайт відкривається за адресою http://localhost:4200.

Тестові облікові записи:

- адміністратор: `admin@automarket.local` / `Admin123!`;
- користувач: `demo@automarket.local` / `Demo123!`.

Тести: `dotnet test` у корені рішення або Run Unit Tests у Rider.

10. Висновки

У ході роботи над груповим проєктом розроблено вебсервіс AutoMarket, що повністю реалізує заявлені сценарії: гість переглядає та шукає оголошення, користувач публікує й редагує власні оголошення та бачить їхні статуси, адміністратор модерує оголошення та керує користувачами.

Серверна частина побудована за шаруватою архітектурою на ASP.NET Core з Entity Framework Core (code-first), що спрощує підтримку та тестування. Клієнтська частина на Angular забезпечує зручний інтерфейс для всіх ролей. Працездатність підтверджена 109 автоматичними тестами.

Можливі напрями розвитку:

- перехід на PostgreSQL для промислової експлуатації;
- завантаження фотографій файлами замість посилань;
- чат між продавцем і покупцем;
- багатомовність інтерфейсу (українська / англійська).